



Payment Transaction

`class odoo.addons.payment.models.payment_transaction.PaymentTransaction`

[\[source\]](#)

`_apply_updates(payment_data)` [\[source\]](#)

Update the transaction based on the payment data received from the provider.

The updates typically include the payment's state, the provider reference, and the selected payment method.

This method should not be called directly; payment data should go through `_process()`.

This method must be overridden by providers to update the transaction based on the payment data.

Note: `self.ensure_one()` from `_process()`

Parameters:

payment_data ([dict](#)) – The payment data sent by the provider.

Returns:

None

`_compute_reference(provider_code, prefix=None, separator='-', **kwargs)`

[\[source\]](#)

Compute a unique reference for the transaction.

The reference corresponds to the prefix if no other transaction with that prefix already exists. Otherwise, it follows the pattern

`{computed_prefix}{separator}{sequence_number}` where:

- `{computed_prefix}` is:
 - The provided custom prefix, if any.
 - The computation result of `_compute_reference_prefix()` if the custom prefix is not filled, but the kwargs are.
 - `'tx-{datetime}'` if neither the custom prefix nor the kwargs are filled.
- `{separator}` is the string that separates the prefix from the sequence number.
- `{sequence_number}` is the next integer in the sequence of references sharing the same prefix. The sequence starts with 1 if there is only one matching reference.

- reference, the full reference will be `example`.
- Given the custom prefix `'example'` which matches the existing reference `'example'`, and the custom separator `'-'`, the full reference will be `'example-1'`.
 - Given the kwargs `{'invoice_ids': [1, 2]}`, the custom separator `'-'` and no custom prefix, the full reference will be `'INV1-INV2'` (or similar) if no existing reference has the same prefix, or `'INV1-INV2-n'` if `n` existing references have the same prefix.

Parameters:

provider_code ([str](#)) – The code of the provider handling the transaction.

prefix ([str](#)) – The custom prefix used to compute the full reference.

separator ([str](#)) – The custom separator used to separate the prefix from the suffix.

kwargs ([dict](#)) – Optional values passed to `_compute_reference_prefix()` if no custom prefix is provided.

Returns:

The unique reference for the transaction.

Return type:

[str](#)

`_compute_reference_prefix(separator, **values)` [\[source\]](#)

Compute the reference prefix from the transaction values.

Note: This method should be called in sudo mode to give access to the documents (invoices, sales orders) referenced in the transaction values.

Parameters:

separator ([str](#)) – The custom separator used to separate parts of the computed reference prefix.

values ([dict](#)) – The transaction values used to compute the reference prefix.

Returns:

The computed reference prefix.

Return type:

[str](#)

`_extract_amount_data(payment_data)` [\[source\]](#)

Extract the amount, currency and rounding precision from the payment data.

Parameters:

payment_data ([dict](#)) – The payment data sent by the provider.

Returns:

The amount data, in the {amount: float, currency_code: str, precision_digits: int} format.

Return type:

[dict](#) | [None](#)

`_extract_reference(provider_code, payment_data)` [\[source\]](#)

Extract the transaction reference from the payment data.

This method must be overridden by providers to extract the reference from the payment data.

Parameters:

provider_code ([str](#)) – The code of the provider handling the transaction.

payment_data ([dict](#)) – The payment data sent by the provider.

Returns:

The transaction reference.

Return type:

[str](#)

`_extract_token_values(payment_data)` [\[source\]](#)

Extract the create values of a token from the payment data.

Providers can override this to supply their own token data based on the payment data.

Note: self.ensure_one() from :meth: `_tokenize`

Parameters:

payment_data ([dict](#)) – Data sent by the provider.

Returns:

Data to create a payment token.

Return type:

[dict](#)

`_get_specific_create_values(provider_code, values)` [\[source\]](#)

Complete the values of the `create` method with provider-specific values.

For a provider to add its own create values, it must overwrite this method and return a dict of values. Provider-specific values take precedence over those of the dict of generic

provider_code ([str](#)) – The code of the provider that handled the transaction.

values ([dict](#)) – The original create values.

Returns:

The dict of provider-specific create values.

Return type:

[dict](#)

`_get_specific_processing_values(processing_values)` [\[source\]](#)

Return a dict of provider-specific values used to process the transaction.

For a provider to add its own processing values, it must overwrite this method and return a dict of provider-specific values based on the generic values returned by this method. Provider-specific values take precedence over those of the dict of generic processing values.

Parameters:

processing_values ([dict](#)) – The generic processing values of the transaction.

Returns:

The dict of provider-specific processing values.

Return type:

[dict](#)

`_get_specific_rendering_values(processing_values)` [\[source\]](#)

Return a dict of provider-specific values used to render the redirect form.

For a provider to add its own rendering values, it must overwrite this method and return a dict of provider-specific values based on the processing values (provider-specific processing values included).

Parameters:

processing_values ([dict](#)) – The processing values of the transaction.

Returns:

The dict of provider-specific rendering values.

Return type:

[dict](#)

`_process(provider_code, payment_data)` [\[source\]](#)

Process the payment data received from the provider and update the transaction.

Returns:

The updated transaction.

Return type:

payment.transaction

`_search_by_reference(provider_code, payment_data)` [\[source\]](#)

Search the transaction based on the payment data.

Parameters:

provider_code ([str](#)) – The code of the provider handling the transaction.

payment_data ([dict](#)) – The payment data sent by the provider.

Returns:

The transaction, if found.

Return type:

payment.transaction

`_send_api_request(method, endpoint, *, params=None, data=None, json=None, **kwargs)` [\[source\]](#)

Send a request to the API.

This method serves as a helper to:

1. Pass the transaction reference to the provider's `_send_api_request()` method.
2. Set the transaction's state to `error` if the request fails, with the exception's message as the `state_message`.

Note: `self.ensure_one()`

Parameters:

method ([str](#)) – The HTTP method of the request.

endpoint ([str](#)) – The endpoint of the API to reach with the request.

params ([dict](#)) – The query string parameters of the request.

data ([dict](#) / [str](#)) – The body of the request.

json ([dict](#)) – The JSON-formatted body of the request.

kwargs ([dict](#)) – Provider-specific data forwarded to the specialized helper methods.

Returns:

The formatted content of the response.

Return type:

`_send_capture_request()` [\[source\]](#)

Request the provider handling the transaction to send a capture request.

For a provider to support authorization, it must override this method and send an API request to capture the payment.

Note: `self.ensure_one()` from `_capture()`

Returns:

None

`_send_payment_request()` [\[source\]](#)

Request the provider handling the transaction to send a token payment request.

This method is exclusively used to make payments by token, which correspond to both the `online_token` and the `offline` transaction's `operation` field.

For a provider to support tokenization, it must override this method and send an API request to make a payment.

Note: `self.ensure_one()` from `_charge_with_token()`

Returns:

None

`_send_refund_request()` [\[source\]](#)

Request the provider handling the transaction to send a refund request.

For a provider to support refunds, it must override this method and send an API request to make a refund.

Note: `self.ensure_one()` from `_refund()`

Returns:

None

`_send_void_request()` [\[source\]](#)

Request the provider handling the transaction to send a void request.

Returns:

None

`_set_authorized(*, state_message=None, extra_allowed_states=())` [\[source\]](#)Update the transactions' state to `authorized`.**Parameters:****`state_message`** ([str](#)) – The reason for setting the transactions in the state `authorized`.**`extra_allowed_states`** ([tuple](#) [[str](#)]) – The extra states that should be considered allowed target states for the source state 'authorized'.**Returns:**

The updated transactions.

Return type:recordset of `payment.transaction`**`_set_canceled(state_message=None, extra_allowed_states=())`** [\[source\]](#)Update the transactions' state to `cancel`.**Parameters:****`state_message`** ([str](#)) – The reason for setting the transactions in the state `cancel`.**`extra_allowed_states`** ([tuple](#) [[str](#)]) – The extra states that should be considered allowed target states for the source state 'canceled'.**Returns:**

The updated transactions.

Return type:recordset of `payment.transaction`**`_set_done(*, state_message=None, extra_allowed_states=())`** [\[source\]](#)Update the transactions' state to `done`.**Parameters:****`state_message`** ([str](#)) – The reason for setting the transactions in the state `done`.**`extra_allowed_states`** ([tuple](#) [[str](#)]) – The extra states that should be considered allowed target states for the source state 'done'.**Returns:**

The updated transactions.

```
_set_error(state_message, extra_allowed_states=(), [source])
```

Update the transactions' state to `error` .

Parameters:

state_message ([str](#)) – The reason for setting the transactions in the state `error` .

extra_allowed_states ([tuple](#) [[str](#)]) – The extra states that should be considered allowed target states for the source state 'error'.

Returns:

The updated transactions.

Return type:

recordset of `payment.transaction`

```
_set_pending(*, state_message=None, extra_allowed_states=()) [source]
```

Update the transactions' state to `pending` .

Parameters:

state_message ([str](#)) – The reason for setting the transactions in the state `pending` .

extra_allowed_states ([tuple](#) [[str](#)]) – The extra states that should be considered allowed target states for the source state 'pending'.

Returns:

The updated transactions.

Return type:

recordset of `payment.transaction`

Get Help

[Contact Support](#)
[Ask the Odoo Community](#)